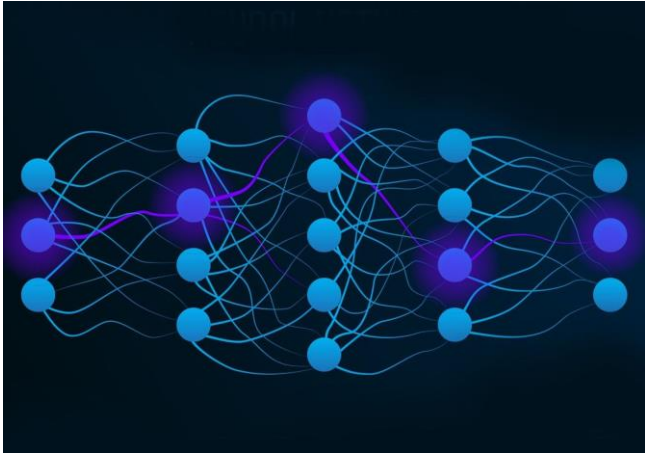




A COMPARATIVE STUDY OF NEXT WORD PREDICTOR MODELS

AUTHOR: HANS-HENRIK HANSEN

Student ID: B01073516

Advanced Topics in NLP – CSCI 4152

A comparative study of Next Word Predictors

Abstract

Next-word prediction is a central task in natural language processing, with applications in text completion, autocorrect, and conversational AI. This study compares three approaches, statistical N-gram models, LSTM networks, and pretrained GPT-2 transformers, using a Wikipedia-derived corpus.

The dataset was preprocessed to remove noise and tokenized, with a vocabulary limited to the 10,000 most frequent tokens, due to computational resource constraints.

Models were evaluated using perplexity and Top-K accuracy to assess their ability to predict the next word in a sequence.

Results show that the N-gram model performs poorly due to data sparsity and limited context, while the LSTM captures longer-range dependencies and improves prediction despite constraints on training epochs and vocabulary size. The pretrained GPT-2 model achieves the best performance, highlighting the benefits of transformer architectures and large-scale pretraining.

Overall, the study illustrates the trade-offs between computational efficiency, model complexity, and predictive accuracy, and demonstrates the practical value of pretrained language models for next-word prediction.

Contents

A comparative study of Next Word Predictors	1
Abstract.....	1
1. Introduction	3
2. Research problem	3
3. Related work	4
4. Methods	4
4.1. Data Description	4
4.2. Preprocessing & Cleaning	5
4.3. Metrics used for evaluation	6
4.4. Models used for training.....	8
4.4.1. Baseline model.....	8
4.4.2. LSTM.....	8
4.4.3. GPT-2.....	9
5. Analysis results	10
5.1. Comparative analysis of performance	10
6. Conclusion	11
7. References	12
8. Appendices	13

1. Introduction

Predicting the next word in a sequence is a fundamental task in natural language processing (NLP) with applications ranging from text completion and autocorrect to conversational AI and language modeling. Traditional methods, such as n-gram models, rely on counting word co-occurrences within a fixed context window and provide a straightforward way to estimate the likelihood of the next word. However, these approaches are limited by their inability to capture long-range dependencies and their sensitivity to data sparsity, which often results in poor performance on large-vocabulary corpora.

Recent advances in deep learning, particularly recurrent neural networks (RNNs) and long short-term memory (LSTM) networks, have enabled models to learn richer contextual representations from text. Unlike n-gram models, LSTMs can capture dependencies over longer sequences, making them more suitable for tasks that require understanding of syntax and semantics. Furthermore, the emergence of pre-trained transformer-based language models, such as GPT-2 and its variants, has significantly improved next-word prediction by leveraging large-scale unsupervised pretraining on massive text corpora.

In this project, we explore the task of next-word prediction using three distinct approaches: a baseline n-gram model, a custom LSTM network trained from scratch, and a pretrained GPT-2 model. We construct a dataset from a Wikipedia-derived corpus and preprocess it to create sequences suitable for both n-gram and neural models. By comparing the models using metrics such as Top-K accuracy and perplexity, we aim to quantify the improvements gained through neural architecture and pretrained models over traditional statistical methods. Additionally, this project illustrates the trade-offs between training models from scratch versus leveraging pretrained models in terms of computational efficiency, prediction quality, and practical usability.

Through this comparative analysis, the study provides insights into the strengths and limitations of each approach and highlights the value of pretrained language models for next-word prediction. The results serve as both an educational demonstration of NLP techniques and a practical demonstration of how to implement text-prediction systems.

2. Research problem

Despite the growing popularity of LLMs, there is limited empirical comparison of their performance relative to classical methods and small-scale neural networks on controlled corpora. Understanding these differences is crucial for researchers and practitioners to

choose appropriate models based on available resources, desired performance, and application constraints. This leads to the following problem formulation:

How do N-gram, LSTM, and pretrained GPT-2 models compare in terms of top-k performance, and perplexity for next-word prediction on a representative English text corpus?

3. Related work

Next-word prediction has been a central task in natural language processing for decades. Early approaches relied heavily on statistical language models, particularly N-gram models, which estimate the probability of a word given a fixed context of preceding words (Kneser & Ney, 1995). These models are simple and interpretable but suffer from data sparsity and cannot capture long-range dependencies in text, limiting their predictive performance on longer or more complex sequences.

With the rise of deep learning, recurrent neural networks (RNNs) and their variants, such as LSTMs (Hochreiter & Schmidhuber, 1997), became popular for sequence modeling tasks. LSTMs address the vanishing gradient problem and can theoretically capture dependencies over longer sequences. Studies have shown that LSTMs outperform N-gram models in next-word prediction and language modeling benchmarks (Sundermeyer et al., 2012). However, LSTMs require substantial training data and computational resources, and their performance is sensitive to hyperparameter choices and sequence length.

More recently, pretrained transformer-based models such as GPT-2 (Radford et al., 2019) leverage large-scale unsupervised pretraining to learn rich contextual representations. These models achieve state-of-the-art performance on next-word prediction and related language tasks. While their predictive accuracy is impressive, they require considerable memory and processing power, and their behavior can be difficult to interpret compared to simpler models like N-grams or LSTM.

4. Methods

4.1. Data Description

The wikitext-2 datasets comprise over 100 million tokens and all of these has been extracted from Wikipedia articles that has been verified as good and featured on Wikipedia. The articles are in their full form, and it makes the dataset excellent for models that benefits from seeing long term dependencies (*Mindchain/Wikitext2 · Datasets at Hugging Face*, 2001).

4.2. Preprocessing & Cleaning

Cleaning the raw data

The data has been imported using the Huggingface API, and predefined subsets of the data have been utilized to organize the data into a training, test and validation set, respectively. The training subset contains a total of 36.718 rows, where each row represents a full article. The test subset contains a total of 4358 rows, and the validation set contains a total of 3760 rows.

The data in its raw form contains a large number of empty lines between the words in the articles. These has been removed by stripping the data from whitespaces and skipping empty lines, since such empty spaces provide only noise to our models. Furthermore, double punctuation is also seen to be present after many characters such as “ . , ; ! ? ” and such double punctuation has also been removed.

Furthermore, the text contained a lot of special characters such as “= Introduction =”, “<umk>” and “@-@” which we have also removed since we do not expect these provide semantic value for our models and shall only want to draw inference from normal characters in this paper.

The text has then been transformed from row data into one long string, from where two subsets of the data have been created. One containing either list of lists of strings/articles and another one containing the string as a continuous sequence.

Preprocessing the data

Using the Keras library, a tokenizer has been instantiated which turns each word in our data into a token. Importantly, the tokenizer is used only on the training data since we want to avoid data leakage to our test- and validation sets, that could otherwise occur if the tokenizer also sees these subsets of the data. The tokenizer limits the tokens to only contain the 10.000 most frequent tokens due to limited computing resources available during our training. All words in the data that are not recognized as tokens are attributed with a <OOV>. Furthermore, all words has been lowercased before being tokenized to prohibit the words with different capitalizations being tokenized twice.

Note that stopwords has not been removed since we want our model to be able to predict words such as a, is, the, etc. and removing stopwords would inhibit our model from learning these word patterns. Furthermore, we do not lemmatize since we do not want to change the natural form of the words. Our models should be able to distinguish between different forms

of the same words since this is important in an LLM where you want to be able to communicate with it fluently.

For the N-gram model, a vocabulary of tokens has been created, where each token is attributed to an ID that makes a given word identifiable through an ID index.

For the LSTM-model, the list of articles is converted into a list of tokens such that each word in the list is represented by a token ID. Each sequence of token IDs can be considered the input and a target output token is created such that the target token is the token that comes right after the input token in its initial sequence. Furthermore, padding has been used to fill shorter sequences with 0's such that all sequences have the same length.

4.3. Metrics used for evaluation

The metrics used for evaluating the performance of next word predictors which form the basis of the LLMs we know today, differ significantly from the measures that would be used in other classification tasks. In a standard classification task, there are only 1 correct outcome, but in the next word predictor multiple different outcomes can be reasonable, since many different words can be used to express the same meaning and may therefore offer themselves as equally valid outputs. If we were to use standard classification metrics, this assumption would be broken since accuracy, F1-scores, precision and recall all assume that only 1 class is the true label.

Furthermore, the number of classes in a next word predictor are huge (limited to 10.000 in our case) and the accuracy will always approach 0 even if the model is performing excellently.

Predicting the next word is essentially a generative and not discriminate task, and we must therefore put our focus on the quality of the probability distribution of outputs, instead of focusing on a single outcome.

It is here that perplexity enters into the picture. Perplexity is defined from the likelihood of the correct word under the predicted distribution and it evaluates the full probability distribution instead of only the top guess.

The formula for perplexity can be written as follows:

$$\text{Perplexity} = 2^H$$

Where H is equal to the average cross-entropy and can be written as:

$$H = -\frac{1}{N} \sum_{i=1}^N \log_2(P(w_i | w_1, w_2, \dots, w_{i-1}))$$

In this formula N is the total number of words in the dataset, and the $P(\dots)$ is the predicted probability of the i^{th} word given the preceding context (Mukherjee, 2023).

Essentially, lower the perplexity means that the models assigns higher probabilities to correct words, and minimizing the perplexity is therefore how to maximize the performance of the model.

Even though perplexity remains one of the best metrics for evaluating the performance of LLMs, it also has some potential weaknesses that are relevant to mention. Most importantly, perplexity has a lack of subjectivity consideration since it does not account for subjective factors such as writing style, creativity and humor. Predicted words can be correct from a grammatical point of view but the style of the sentence that the predicted word results in can be very bland and lack aforementioned style. Therefore, it is not sufficient to rely on perplexity when building competitive models to be put in production but for the scope of this paper, perplexity remains an excellent measure of performance for our models.

Below is an overview of typically seen perplexity scores observed in different models, and this has been derived from previous NLP studies (Mikolov et al., 2010; Zaremba et al., 2014; Radford et al., 2019; Jurafsky & Martin, 2024). Note that the perplexity scores are highly dependent on the type and size of the datasets, so these are only guiding ranges, not in any way standardized rules.

Model Type	Typical Perplexity	Interpretation
Trigram (3-gram)	100–300	Medium-range performance; limited to short context (2 previous words).
LSTM / RNN	30–150	Captures longer dependencies; performance improves with sequence length and embedding quality.
GPT-2 / DistilGPT-2	20–50	Transformer-based; handles long-range context and subword tokens. Pretrained knowledge lowers PPL.

Furthermore, Top-K has been included as an additional measure to showcase how many of the next true words are within the Top-K of the most likely predictions, expressed as a percentage out of the total amount of true words. Thus, the Top-K measure illustrates the quality of the probability distribution of the model.

Top-1 and top-3 have been included to supplement the perplexity metric for the evaluation of the models trained in this paper.

4.4. Models used for training

4.4.1. Baseline model

As a baseline model to measure the performance of a simple model, an N-gram model that uses additive Laplace smoothing has been built, trained and evaluated. The model uses trigrams ($n=3$), such that each token is comprised of a 2-word context and this is used to estimate the probability of the next word based on the preceding $n-1$ trigrams to capture more context than for example a bigram or unigram model. Using trigrams for a medium sized to large dataset is in principle good practice. Increasing the n -grams would capture more context but suffer from data sparsity given that many sequences of 3+ words would never appear again in the dataset and decreasing the n -grams would not capture enough context, thus, a tri-gram balances the trade off between these two directions well. However, since the token vocabulary has been limited to 10.000 tokens, the trigram was not feasible to implement since the trigrams were often only seen once in the data which significantly damaged the perplexity score of the model. Instead, a bigram model were implanted despite it capturing less context.

It utilizes the vocabulary of 10.000 tokens + 1 for unseen words (<OOV>), creates maximum-likelihood counts based on the counts of tri-grams and corresponding target words, and estimates the probability using Laplace smoothing to avoid zero probabilities that would otherwise occur for words that never appear in the training data. We want to avoid zero probabilities since these can skew our model and diminish the credibility of the perplexity metric.

The implemented n -gram model is computationally very simple and does not require much computational power to train, and it provides a solid baseline for model performance in this paper.

4.4.2. LSTM

To model sequential dependencies beyond the fixed context window of n -gram models, a Long Short-Term Memory (LSTM) network for next-word prediction has been implemented. The network takes fixed-length sequences of tokenized words as input and outputs a probability distribution over the vocabulary for the next word.

The sequence length has been set to 15 due to limited computational resources but is still deemed to contain enough context for our LSTM model to be trained well. The words are projected onto a 32-dimensional embedding layer such that each word is represented by a 32-sized vector. This is to give words a more compact and dense representation which also increases computational efficiency for our model.

Sequences of these embeddings are fed into an LSTM layer, which maintains hidden and cell states across timesteps to capture sequential dependencies. The LSTM outputs a single vector summarizing the entire input sequence, which is passed to a dense softmax layer that produces a probability distribution over the vocabulary for the next-word prediction.

A dropout layer with rate 0.2 is applied after the LSTM to reduce overfitting by randomly zeroing activations during training and the training procedure can be described as follows:

- **Optimizer:** Adam with learning rate 0.001
- **Batch size:** 64 sequences per batch
- **Maximum epochs:** 10 (training was limited by available computational resources)
- **Verbose:** 1 (progress printed per epoch)
- **Early stopping:** Monitors validation loss with a patience of 3 epochs. The best model weights (lowest validation loss) are restored to prevent overfitting.

A conservative learning rate of 0.001 one has been chosen to avoid overstepping during the gradient descent learning process.

The LSTM model has been preferred over other neural networks such as a standard RNN since it works well for capturing contextual information in longer sequences without vanishing/exploding gradients becoming a large problem.

By maintaining memory cells and using gating mechanisms, LSTMs can retain relevant information across many timesteps, enabling the model to capture dependencies that span entire input sequences. This allows the LSTM to exploit longer-range contextual information than trigram models, improving predictive accuracy and generalization. The architecture, combined with learned embeddings, dropout regularization, and early stopping, makes the LSTM both effective and computationally feasible for medium-sized datasets, providing a strong neural baseline for comparison with statistical n-gram models.

4.4.3. GPT-2

To provide a modern neural baseline for next-word prediction, the DistilGPT-2 model has been employed using the Huggingface API, and it is a distilled variant of GPT-2, which is a transformer-based causal language model with 82 million parameters trained on the OpenWebCorpus which is the same as the GPT2 model were trained on. Transformers leverage self-attention mechanisms to capture dependencies across arbitrary sequence lengths, overcoming the limitations of recurrent architectures in modeling long-range context.

The inbuilt AutoTokenizer has been used to tokenize the cleaned dataset since the tokenized data needs to fit a structure that works with the distilGPT-2 model. Each text/article is tokenized and truncated to a maximum of 128 tokens. For each tokenized sequence, a shifted

input-target scheme is used: context_ids = all tokens except the last, target_ids = all tokens except the first, (*AutoModel for Causal LM*, n.d.).

For evaluation, the tokenized sequences are fed into the model in a shifted context-target manner, allowing computation of top-1 and top-k accuracy as well as perplexity. This model serves as a modern neural baseline for next-word prediction, complementing trigram and LSTM models in our experiments, (*Distilbert/Distilgpt2 · Hugging Face*, 2024).

5. Analysis results

5.1. Comparative analysis of performance

The 3-models perform differently, and their performance can be summarized as below:

Models	Top-1 accucary	Top-3 accuracy	Perplexity
N-gram	0.2066	0.3448	3426
LSTM	0.2214	0.3607	159.02
GPT-2	0.2943	0.4437	69.79

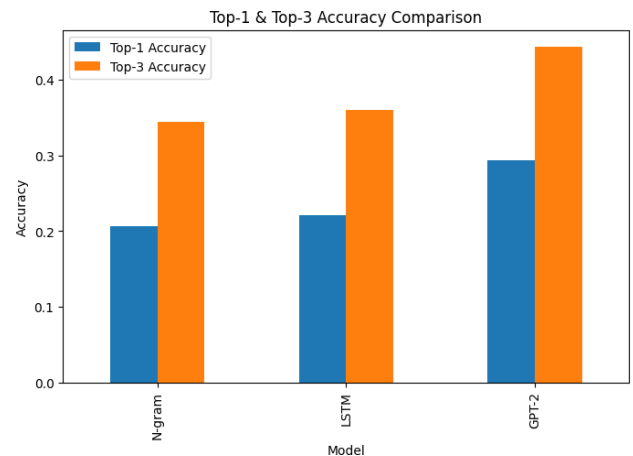
From this we see that the N-gram model has a perplexity score of 3426 which makes it practically useless. There are several possible explanations for the poor performance of the n-gram model, and one is that the vocabulary has been limited to 10.000 words and many of the bigrams will most likely only appear once or very few times in the corpus when the dataset is not larger than it is. Initially, the n-gram model was run with trigrams, and the perplexity was at an astounding 28000 and reducing it to a bigram model improved it significantly which supports this explanation for its poor performance. Improving the bigram model requires expanding the vocabulary to increase the chance that more bigrams will be seen more than once in the training data.

For the LSTM model we see an overall perplexity score of 159.02 which is relatively poor performance considering the typical performance of LSTM models that lie within the range of 30 to 150. However, this is most likely due to the number of epochs of the LSTM having been reduced to only 10, and the vocabulary having been limited to only 10.000 tokens. Due to very long training times, the model has also only been fine tuned to a very low extent. If more computing resources had been available, it would have been highly interesting to explore how increasing the epochs and vocabulary size and finetuning the model through hyperparameter adjustments would have impacted the model's performance.

If we consider the GPT-2 model, we see that it achieved a perplexity of 69.79 on the test data evaluation which is a significant improvement from the LSTM and a clear demonstration that transformer-based architecture that has been pretrained on a very large dataset will significantly improve the performance of a next word predictor. Furthermore, using a

pretrained model is the only sensible approach when it comes to building and training a next word predictor if somewhat satisfactory results are required given the significant computational complexity of training these models and the restraints on computing power, that someone who does not have access to state-of-the-art GPUs, experience.

To the right, we also see the top-1 & top-3 accuracy which also clearly demonstrates that the quality of the probability distribution of GPT-2 is superior to that of the LSTM and the N-gram model.



6. Conclusion

This study compared three approaches to next-word prediction: N-gram, LSTM, and pretrained GPT-2 models. The N-gram model performed poorly (perplexity 3426) due to data sparsity and limited context, highlighting the limitations of traditional statistical methods. The LSTM improved performance (perplexity 159.02) by capturing longer-range dependencies, though its results were constrained by limited training, vocabulary size, and computational resources.

The pretrained GPT-2 model achieved the best results (perplexity 69.79), demonstrating the advantages of transformer architectures and large-scale pretraining. These findings illustrate a clear hierarchy: N-grams are simple but limited, LSTMs capture sequence patterns effectively, and pretrained transformers provide state-of-the-art predictive power.

Overall, the study emphasizes the value of pretrained models for practical next-word prediction and suggests that further improvements could be achieved through larger vocabularies, extended training, and fine-tuning.

7. References

mindchain/wikitext2 · Datasets at Hugging Face. (2001, March 10).

<https://huggingface.co/datasets/mindchain/wikitext2>

Team, K. (n.d.). *Keras documentation: Tokenizer*.

https://keras.io/keras_hub/api/tokenizers/tokenizer/

distilbert/distilgpt2 · Hugging Face. (2024, May 17). <https://huggingface.co/distilbert/distilgpt2>

AutoModel for Causal LM. (n.d.). HuggingFace. Retrieved December 3, 2025, from

https://huggingface.co/docs/transformers/model_doc/auto#transformers.AutoModelForCausalLM

GPT-2. (n.d.). https://huggingface.co/docs/transformers/model_doc/gpt2

Mukherjee, S. (2023, June 15). Unveiling Perplexity: Measuring Success of LLMs and Generative AI Models. *Medium*. <https://ramblersm.medium.com/the-significance-of-perplexity-in-evaluating-llms-and-generative-ai-62e290e791bc>

Hankare, O. (2024, March 8). Exploring N-Grams: The building blocks of Natural Language Understanding. *Medium*. <https://ompramod.medium.com/exploring-n-grams-the-building-blocks-of-natural-language-understanding-b9b90dffe0a2>

Mikolov, T., Karafiát, M., Burget, L., Černocký, J., & Khudanpur, S. (2010). *Recurrent neural network based language model*. In *Proceedings of the 11th Annual Conference of the International Speech Communication Association (INTERSPEECH 2010)* (pp. 1045–1048)

Zaremba, W., Sutskever, I., & Vinyals, O. (2014). *Recurrent Neural Network Regularization*. arXiv preprint arXiv:1409.2329

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners*. OpenAI Blog.

Kneser, R., & Ney, H. (1995). Improved backing-off for n-gram language modeling. *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 1, 181–184. <https://doi.org/10.1109/ICASSP.1995.479391>

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>

Sundermeyer, M., Ney, H., & Schluter, R. (2012). LSTM neural networks for language modeling. *Proceedings of INTERSPEECH 2012*, 194–197. <https://doi.org/10.21437/Interspeech.2012-77>

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. OpenAI. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

8. Appendices

Appendix Name	Description
Project_Final.ipynb	A jupyter notebook that contains all relevant code for data cleaning & preprocessing, building & training models and evaluation of the models
Lstm_model.h5	The trained lstm model
Ngram_model.pkl	The trained ngram model
Ngram_model.py	A class for the ngram model that is used for streamlit UI
Streamlit_run.py	Code using the streamlit library for creating a UI running locally in the browser where top-k predictions for each model can be inferred by providing a prompt